VOUCHSAFE × IITM · SEQUENTIAL MATCHING PROBLEM · ROUND 2

Supply-Aware Sequential Matching

Fewer people left waiting, with the score held at the baseline and the evidence stated honestly

Kashmira Matale · Team sol · 24f3002508@ds.study.iitm.ac.in

Brief solution summary

1. Supply sets the ceiling

The supplied greedy baseline already introduces about 78% of the truly feasible pairs, so allocation has little left to gain.

2. One change that held

A waiting priority: members who have waited longest for a first introduction get a higher edge weight in the daily matching.

3. What it achieved

About 1.2 fewer members left waiting per episode and one day less waiting, with no provable change in MSMI.

MSMI is under one event per episode, so no MSMI difference in this study is separable from noise.

What the score measures

MSMI = completed meeting plus mutual second-meeting intention, counted per 100 arrived members. A pair must survive five stages:

1. Feasible pair
Hard constraints pass (age, zone, schedule, ...)

2. Introduction
The policy picks which feasible pairs to introduce

3. Two Yes answers
Both accept (stochastic, delayed)

4. Date in 30 days
A date actually happens

5. Two on-time Yes
Both want a second meeting

Only about 2% of introductions complete the chain, so an episode of 200 members has 0 to 3 events. The policy controls only stages 1 and 2 (orange); feedback arrives 1 to 14+ days late.

Where the headroom is

78%

of truly feasible pairs are already introduced by the supplied greedy baseline

~31%

of members decline a hard field and can never be matched

~118

feasible pairs per episode in the standard world (about 21 in sparse geography)

Conclusion from Round 1: feasible supply, not allocation, sets the ceiling. The remaining levers are using pairs earlier, serving everyone who can be served, and spending fewer clarification units.

What greedy already does with feasible pairs



Introduced 78%

Member left 10%

Paused after mutual interest 5%

Unused (timing) 7%

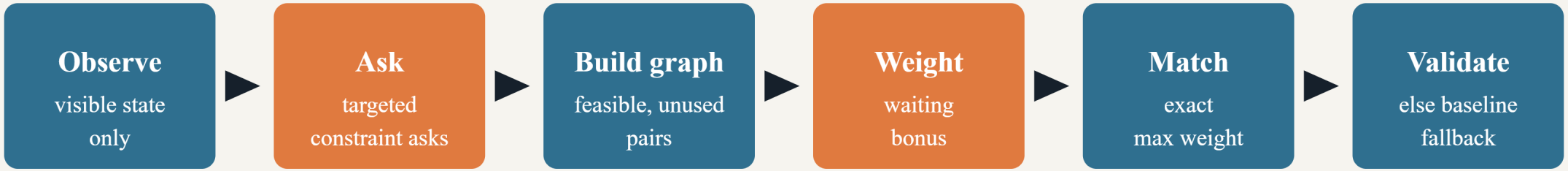
311 truly feasible pairs, supplied greedy baseline, 3 seeds, standard scenario (Round 1)

Greedy already introduces most feasible pairs, so the headroom is small: the 10% lost when members leave is what using pairs earlier can recover.

Methodology and algorithms

Step	What the policy does
Eligibility	Supplied rules only; never overridden; declined members are never matched
Clarification	Targeted constraint asks for members with a feasible partner; soft-field asks off
Allocation	Exact maximum-weight matching of the day's feasible pairs (components up to 18 members)
Waiting priority	Edge weight × (1 + 2·(sa + sb)), s = days since arrival (cap 30) for never-introduced members
Prior	Same for every member in the default; learned Beta propensities kept as an option
Safety	Every batch re-validated; falls back to the supplied baseline on any error

One decision day, step by step



Orange steps are where the policy makes its own choices: who to ask and how to weight pairs. Every batch is validated against the supplied rules before it is returned.

The waiting rule in detail

Part	Definition
Who is "waiting"	An available member never yet introduced; s = days since arrival, capped at 30, divided by 30 (so 0 to 1)
Edge weight	$w = 1 \times (1 + 2 \cdot (sa + sb))$, times 1.5 if both observed relationship goals match
What it does	A pair with two long-waiting members can weigh up to 5× a pair of new arrivals, so the matching serves them first
Why	Members can leave, acceptance falls after day 35 in the drift world, and late dates are worth less

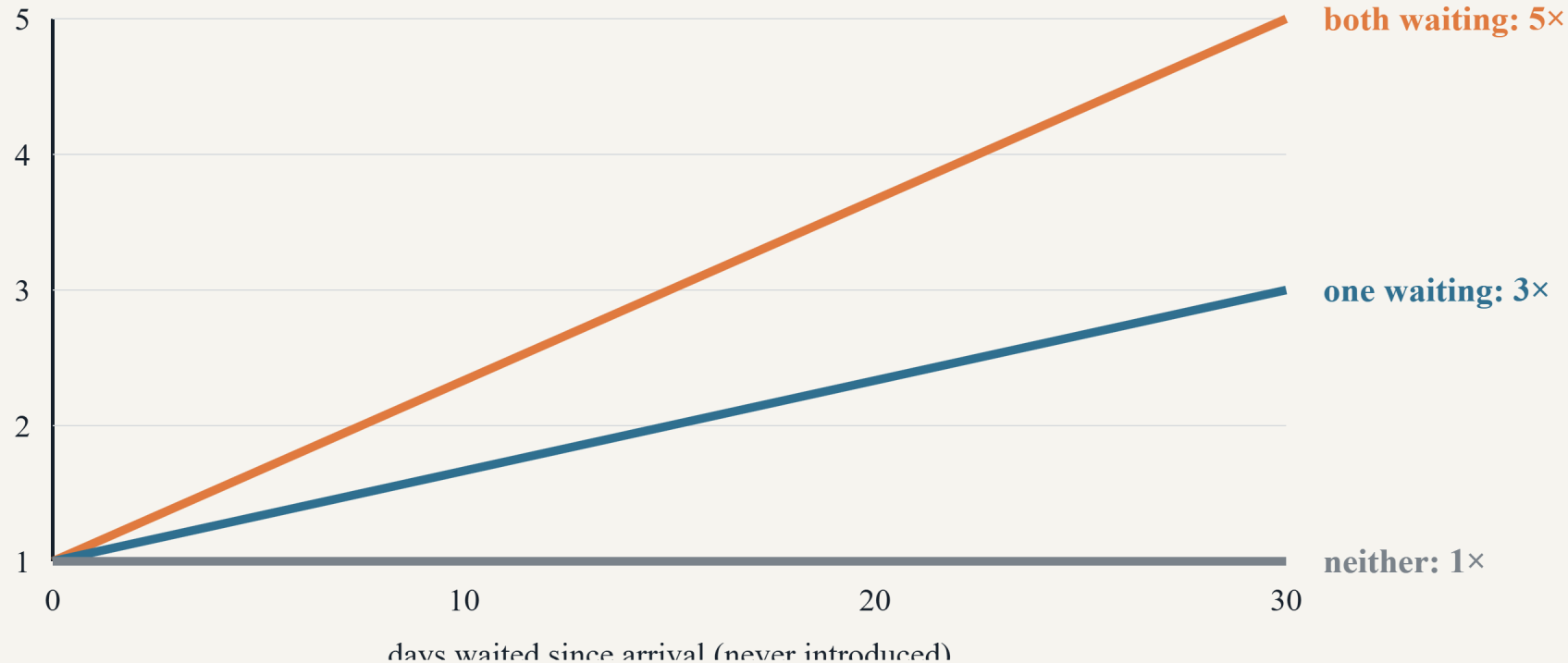
Weight sweep

$\lambda = 1$, 2 and 4 gave the same left-waiting result (−1.06) on the selection seeds, so the result is not sensitive to the exact weight.

Honest caveat

The form of the bonus is my own design, not tuned on held-out seeds; $\lambda = 2$ was fixed in advance.

How the waiting weight grows



Edge weight = 1 + 2·(s_a + s_b), with s = days waited / 30, capped at 30 days. Weights are relative, so only the ratios matter to the matching.

Exact daily matching

Detail	What was done
Graph	Nodes: members available today. Edges: pairs that pass supplied eligibility and were not introduced before
Solver	Split into connected components; components of up to 18 members solved exactly by memoised recursion; larger ones use greedy by weight
Correctness	A unit test checks the exact solver against brute force on random small graphs
vs the Round 1 prototype	Total matching weight equals networkx blossom on 333 of 337 days (98.8%); identical pair sets on 285 (84.6%, ties); max gap 5.8%
Cost	About 0.13 s per call at 200 members; 0.8 s per call including container startup, limit 10 s

Greedy allocation with the same waiting priority does nearly as well on who is left waiting (−1.14 versus −1.24), so exact matching is kept because it is cheap, not because it is required.

Clarification (ask) policy

Detail	What the policy does
Budget	12 ask units per day; the constraints bundle costs 3 units, a soft-field ask costs 1
Who is asked	Available members with missing hard fields, no declined hard field, and at least one optimistically feasible partner
Ranking	By a weighted partner count: 1.0 for partners already fully known, 0.4 otherwise
Declined answers	Never asked again and never matched; about a third of members are blocked this way
Soft asks	Off in the default: about +51 units for no gain on any outcome

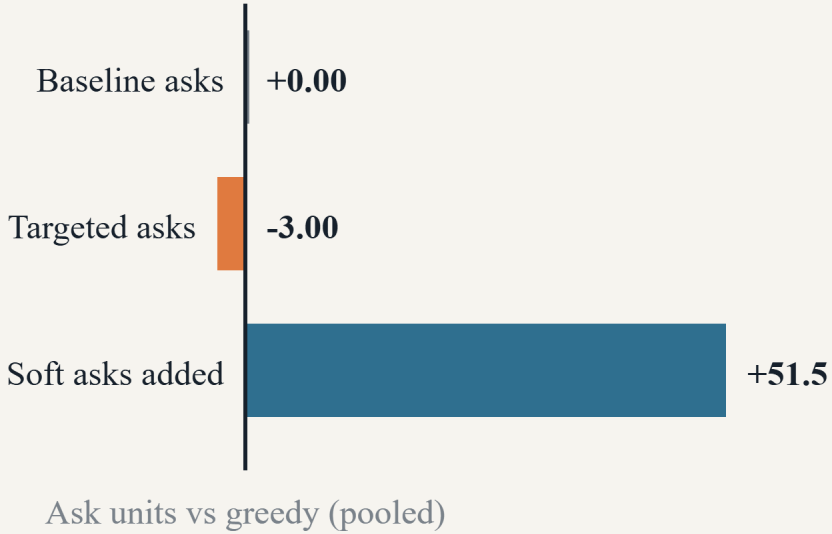
Targeted vs baseline asks

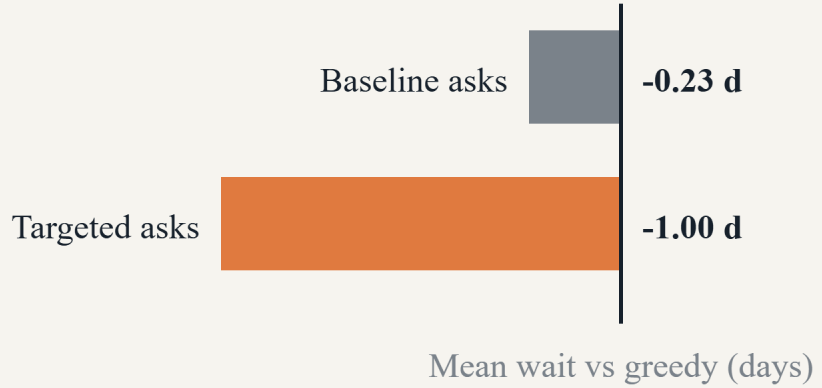
Cost about 3 units less (about 2%), but mean wait is −1.00 days versus −0.23 with baseline asks.

No asks at all

37.9 more members left waiting [30.7, 44.4] and about 5 fewer mutual acceptances than greedy.

What the ask policy buys





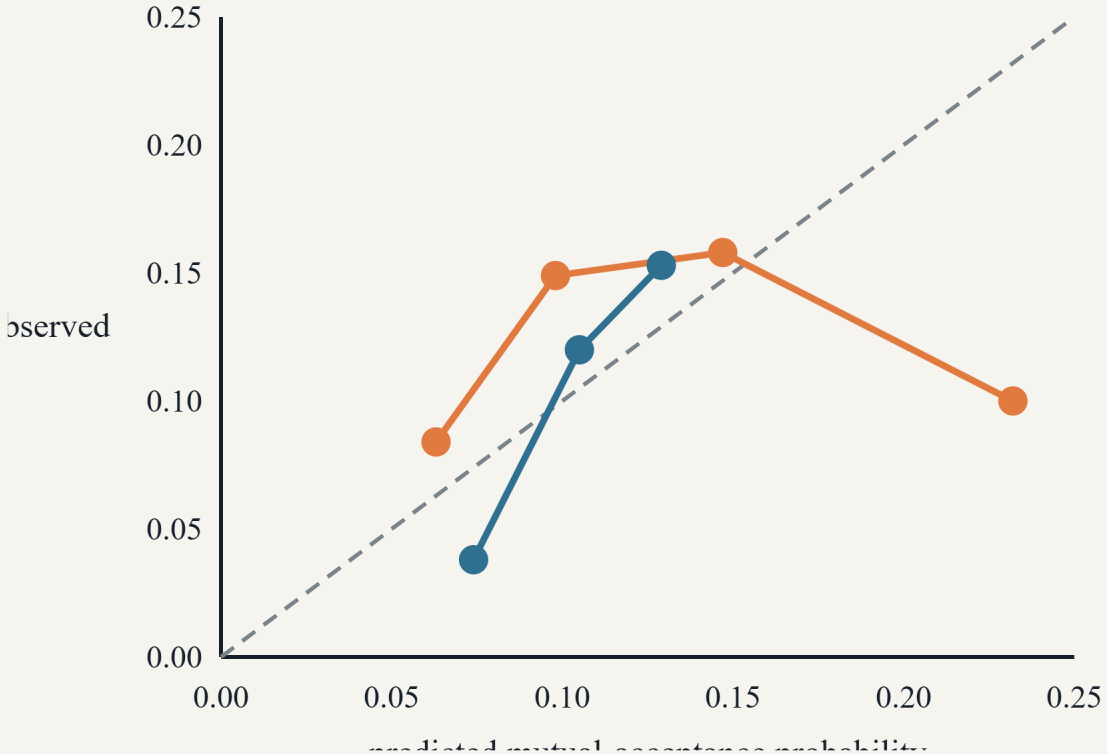
Targeted asks cost about the same units as baseline asks (about 3 fewer) but cut the wait to a first introduction by four times as much. Soft-field asks add 51.5 units with no gain.

Propensities and calibration

Item	Detail
Member score	f = P(responds) × P(says Yes) × P(second-meeting Yes), each a Beta posterior mean (successes + κ·p0) / (trials + κ)
Hand-set priors (p0, κ)	(0.80, 4), (0.40, 4), (0.53, 3)
Fitted priors (stage A only)	(0.78, 12), (0.45, 8), (0.66, 50, capped); members differ less than assumed
Mean prediction vs observed	Hand-set 0.10, fitted 0.118, observed 0.137 mutual acceptance rate
Quality (fresh seeds)	Brier 0.119 / 0.118 versus 0.118 for the base rate; AUC 0.554 / 0.549

Each member has only 1 to 3 observations per episode, so estimates stay near the prior. That is why learning shows little: the ranking signal is too weak to use.

Calibration of the propensity model



hand-set priors

fitted priors

dashed line = perfect calibration

Fresh seeds: Brier 0.119 / 0.118

(base rate alone: 0.118)

AUC 0.554 / 0.549

Mean predicted 0.10 / 0.118, observed 0.137

Fitting the priors moves predictions closer to the observed rate, but the Brier score stays at the base rate and the ranking signal is weak, so learning cannot help the matching much.

How it was evaluated

Item	Details
Baselines	Supplied greedy, no-clarification and random-feasible; Round 1 prototype and build
Metrics	MSMI per 100, mutual acceptances, dates, coverage, ask cost, members left waiting, mean wait, calibration
Protocol	Paired episodes on identical worlds; 95% bootstrap intervals over episode differences
Stages	A: 54 episodes (selection) · C: 36 fresh episodes · stress: 20 episodes in 5 changed worlds
Checks	9 unit tests; official evaluator and Docker container, seed 101, all six scenarios valid

Evaluation protocol in detail

Item	Detail
Worlds	200 members, 60 arrival days plus 40 settling days; six scenarios: development, sparse, cold start, delayed, shift, drift
Seeds	Stage A 3001–3009 (selection, 54 episodes); stage C 6001–6006 (fresh, 36); stress 7001–7004 (20)
Pairing	Every policy runs on the identical world; statistics are on per-episode differences
Intervals	95% percentile bootstrap, 2000 resamples over episodes
No leakage	The policy sees only state observed by the day; priors fitted on stage A only; stage C untouched during selection
Left waiting	Members with a truly feasible partner who never got an introduction; uses simulator truth after the fact, a diagnostic only

Mean wait is days from arrival to first introduction for members who were served. The oracle policy uses simulator truth only inside the evaluation harness, as an upper bound.

How the evidence was built



Fresh seeds were not touched during selection, so stage C is an honest test. Results from A and C are then pooled for 90 paired episodes.

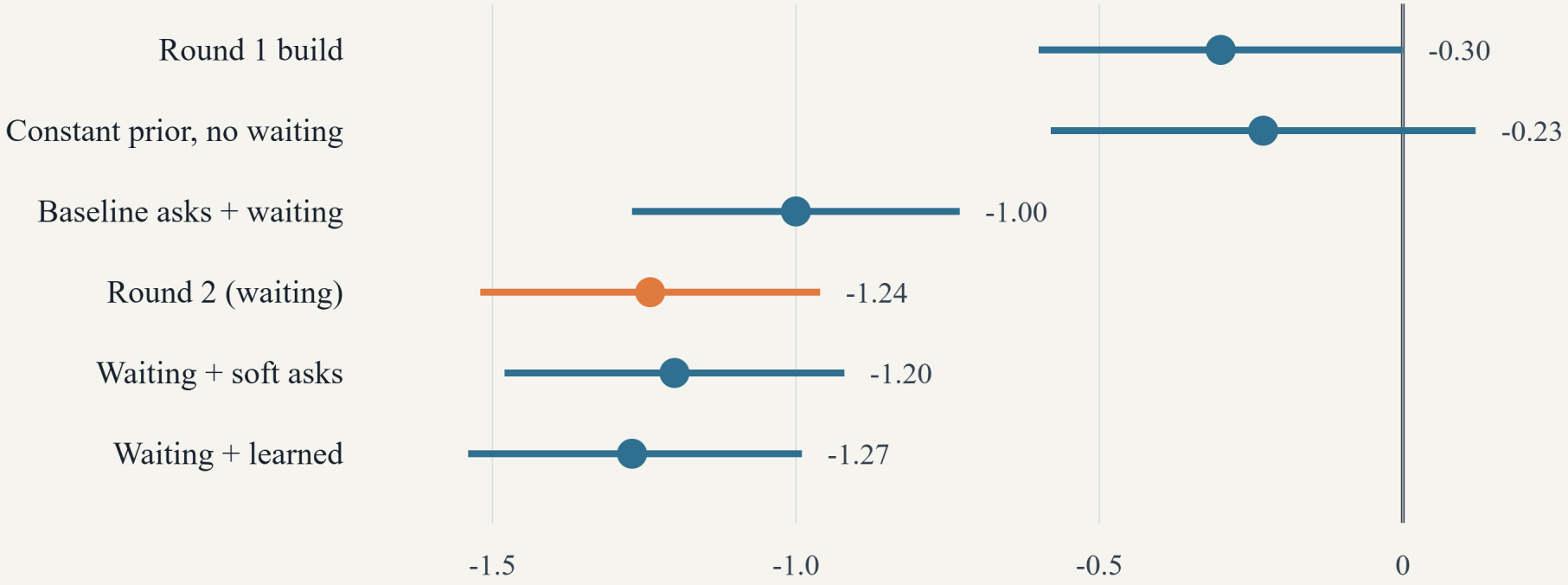
Results: pooled, 90 paired episodes vs greedy

Policy	Left waiting	Mean wait (days)	MSMI / episode	Mutual acc.
Round 1 build	−0.30 [−0.60, 0.00]	−0.80 [−0.99, −0.60]	+0.11 [−0.16, +0.38]	+0.66 [+0.02, +1.31]
Round 2 (waiting)	−1.24 [−1.52, −0.96]	−1.00 [−1.18, −0.81]	+0.08 [−0.19, +0.37]	−0.07 [−0.77, +0.63]
Waiting + learned	−1.27 [−1.54, −0.99]	−0.95 [−1.13, −0.77]	+0.21 [−0.06, +0.47]	+0.09 [−0.53, +0.71]
Waiting + soft asks	−1.20 [−1.48, −0.92]	−1.02 [−1.21, −0.84]	+0.01 [−0.24, +0.30]	−0.32 [−1.04, +0.42]

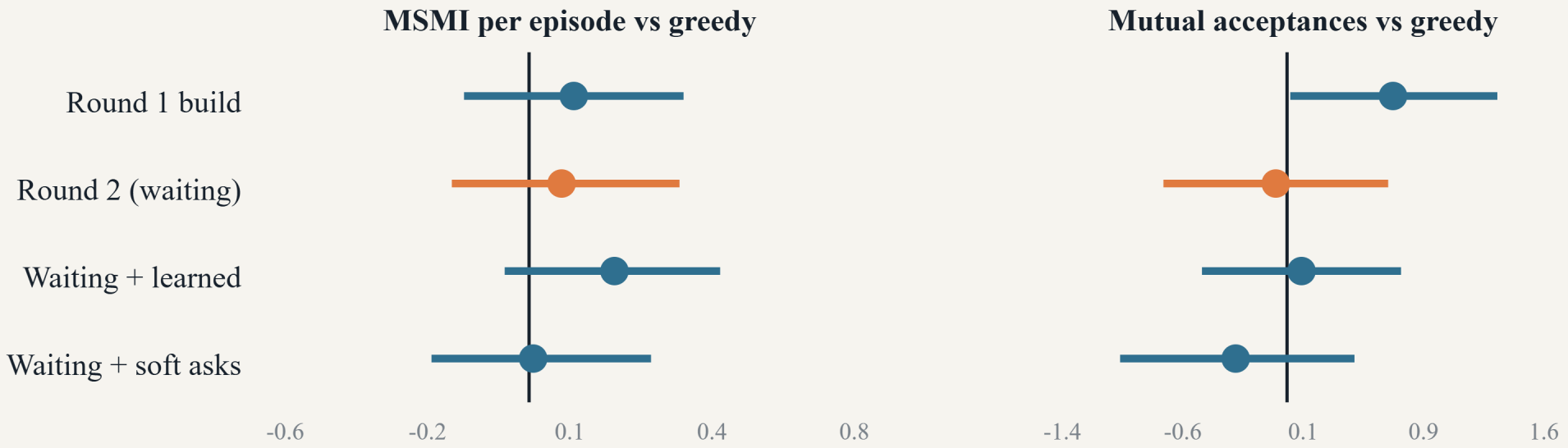
Waiting gain held on fresh seeds. Soft asks cost about 51 extra ask units for no gain. An oracle with true probabilities showed no waiting gain either.

Who gets left waiting

Change per episode vs greedy, with 95% intervals (pooled 90 episodes). Left of zero is better.



Outcomes vs greedy: what separates from zero



Almost every interval crosses zero: only the Round 1 build shows a mutual-acceptance gain, and none of the policies shows a provable MSMI gain. The waiting policies earn their value on who is left waiting.

Changing conditions and calibration

+37.9

more members left waiting if all clarification asks are removed [30.7, 44.4]

+51

extra ask units from soft-field asks, with no gain on any outcome

0.55

AUC of the learned mutual-acceptance prediction; Brier score equals the base rate

Five stress worlds (heavy missingness, low response, response dropping mid-episode, bimodal response, strong acceptance differences), 4 episodes each. Targeted asks stay essential. Learning helps at most where members truly differ (+0.25 MSMI [0.00, +0.75] in those worlds), and hurts slightly when response drops mid-episode.

The five stress worlds

World	How the world was changed	Takeaway (4 episodes each)
Heavy missingness	70% of answers removed, 15% of the rest declined	Degenerate: greedy coverage 0.04
Low response	Response rates × 0.6	Asks remain essential
Response drop	Response rate halved at day 30	Learning slightly hurts (−0.25 MSMI)
Bimodal response	Response rate 0.2 or 0.95	Learning plausible (+0.25 MSMI)
Strong acceptance bias	Member bias sd 1.6	Learning plausible (+0.25 MSMI)

Pooled over the 20 stress episodes: the waiting rule still reduces members left waiting (−0.30 [−0.55, −0.05]); learning on top of the default is +0.05 MSMI [−0.10, +0.20]; baseline asks give +0.65 mutual acceptances for +2.6 ask units, all within noise.

Reproducing Round 1, and the trade-off

Comparison	Result
Prototype vs stdlib build (fresh seeds)	MSMI −0.28 [−0.69, +0.14]; mutual acceptances +0.22 [−0.44, +0.97]; dates +0.89 [+0.17, +1.67]; +54 ask units
Round 1 build vs Round 2 (pooled)	Round 1 has +0.72 mutual acceptances [+0.23, +1.20] and +0.67 dates [+0.20, +1.16], but 0.94 more members left waiting
Round 2 vs greedy, fresh seeds only	Dates −1.17 [−2.50, −0.03], mutual acceptances −0.81 [−1.75, +0.06]; neutral only when pooled with selection seeds
Waiting + learned vs default	Pooled MSMI +0.13 [+0.02, +0.26], not confirmed on fresh seeds alone

The Round 2 policy is not a free improvement. It trades some mutual acceptances for fairer service, and I state this in the report.

Limitations and improvements from Round 1

Round 1 expectations revisited

Targeted asks save little cost (about 3 units) but shorten waits. Soft asks are not supported. Learned propensities are weak (AUC about 0.55). Greedy allocation does nearly as well as exact matching once waiting is used.

Limitations

Under one MSMI event per episode. Stress runs have only 4 episodes each. The waiting weight is untuned. The waiting rule gives up some of the Round 1 mutual-acceptance gain. Intervals are not corrected for multiple comparisons.

Next: 30+ fresh seeds, a pre-planned test of waiting + learned propensities, held-out tuning of the waiting weight.

Validity and reproducibility

Check	Result
Unit tests	9 pass: valid episodes, determinism, exact matching vs brute force, fallback on error, waiting term, constant prior, no declined asks
Official evaluator, seed 101	All six scenarios valid; MSMI 0.5, 1.0, 0.5, 0.5, 0.5 and 0.0 per 100 by scenario; overall 0.5, coverage 0.4175, ask cost 199
Docker container	Same result; about 98 s per episode, roughly 0.8 s per call including startup, against the 10 s limit
Reproduce everything	scripts/run_round2.ps1 (about 1 hour on 1 core); results saved in results/*.jsonl; priors fitted from seeds 3001–3009

Standard library only for the policy; deterministic; no training; no persistent files. Seed 101 is one seed with about one event per episode, so 0.5 is a validity check, not a ranking prediction.

Likely questions

Question	Short answer
Why not just use greedy?	Greedy already covers most supply, but it leaves members waiting; the waiting priority fixes that without losing score
Is the MSMI gain real?	No claim: under one event per episode, no MSMI difference separates from zero
Why drop learned propensities?	Weak ranking signal (AUC about 0.55) and no confirmed gain; it stays as an option
What would you do next?	30+ fresh seeds, a pre-planned test of waiting + learning, tuning the waiting weight on held-out seeds
What did not work?	Soft-field asks, learned propensities alone, and exact matching beyond greedy once waiting is used

Setup, data and conclusion

Setup

Python 3.10+. The policy uses only the standard library; analysis adds numpy, scipy, matplotlib, networkx.
python -m unittest -v
python evaluate.py --seeds 101 --variants all
Docker: build the image, then evaluate with --image.

Data: public synthetic simulator only (kit release 1.0.0); no real people.

Conclusion

The policy keeps the score at the greedy level and serves people more fairly. It does not prove a higher MSMI. Every official and Docker episode was valid.

Code, results and report: github.com/Kashmiraforgta/rj-hackathon-iitm-ml

AI assistance: Claude (Anthropic), for coding and report writing